

Java Persistence Api (JPA)

Todas las respuestas están sacadas de la siguiente fuente:

<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html>

¿Qué es JPA?

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#introduction>

Al comienzo de la sección, podemos notar que Jakarta API se basa en el manejo de persistencia y mapeo objeto-relacional. El objetivo principal es proveer un estándar de mapeo objeto-relacional para aquellos desarrolladores de aplicaciones Java usando un modelo de dominio Java para administrar los datos almacenados en una BDR.

¿Cuáles son las principales características de JPA?

Una de las características más relevantes es que JPA brinda un **estándar de mapeo** de un modelo de dominio Java a guardar datos en una BDR, usando actualmente anotaciones o antiguamente un archivo XML como metadata.

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#introduction>

<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a13915>

Otra característica importante sobre JPA es que posee **entidades**, es decir, objetos persistentes y livianos que soportan herencia, asociaciones polimórficas y consultas polimórficas.

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#entities>

Otra característica que posee JPA es EntityManager, una interfaz que define los métodos utilizados para interactuar con su contexto de persistencia. La API de EntityManager se utiliza para crear y eliminar instancias de entidades persistentes, para buscar entidades persistentes por clave primaria y para realizar consultas sobre los tipos de entidades persistentes.

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#overview>

¿Qué significa que una clase sea de tipo Entidad?

Que una clase sea de tipo Entidad significa que representan información persistente.

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#entities>

¿Cuáles son las anotaciones que se pueden utilizar en una clase de tipo entidad?

La anotación principal que se puede utilizar en una clase de tipo entidad es: **@Entity**. Lo que especifica es que esa clase es una Entidad.

La anotación **@EntityListeners** permite conectar una entidad con clases externas (listeners) para capturar eventos de su ciclo de vida (como guardar, actualizar o borrar). Puede ser aplicada a una clase Entidad o una superclase mapeada.

La anotación **@ExcludeSuperclassListeners** se usa en una clase de entidad para evitar que se ejecuten los métodos oyentes (**@EntityListeners**) definidos en sus clases padre o superclases.

La anotación **@ExcludeDefaultListeners** se usa a nivel de clase para indicar que una entidad debe omitir la ejecución de cualquier oyente (listener) predeterminado.

La anotación **@NamedEntityGraph** define un grafo de entidades. Esta anotación debe aplicarse a la entidad raíz del grafo y especifica los límites del grafo de atributos y entidades asociadas que se recuperan cuando se ejecuta una operación que obtiene una o varias instancias de la entidad raíz.

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#metadata-annotations>

La anotación **@id** define que un atributo de la clase será considerado clave primaria al crearse la tabla.

La anotación **@Column** permite definir características de un atributo que luego se representará en la Base de Datos.

La anotación **@table** permite definir características de la clase que luego se representarán en la Base de Datos.

Respuesta encontrada en: las diapositivas de la clase.

¿Cuáles son las anotaciones que se pueden utilizar para validar información en un atributo de JPA?

La anotación **@temporal** se utiliza para atributos de tipo date donde se puede especificar como será almacenado el tipo de dato en la base de datos.

Temporaltype.date: guarda solo fecha.

Temporaltype.time: guarda solo hora.

Temporaltype.timestamp: guarda fecha y hora

La anotación **@Transient** se utiliza para anotar una propiedad o campo de una clase de entidad, superclase mapeada o clase incrustable. Indica que la propiedad o el campo no es persistente.

Respuesta encontrada en: las diapositivas y

<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#transient-annotation>

¿Cuáles son las relaciones entre clase que se pueden anotar en JPA para ser mapeada a una base de datos? ¿Dar un ejemplo sencillo de cada una de las relaciones y realizar el DER para cada ejemplo?

- **@OneToOne** (<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#onetoone-annotation>)

EJEMPLO:

@Entity

```
public class Empleado {
```

```
    @Id @GeneratedValue
```

```
    private Long id;
```

```
    private String nombre;
```

```
    @OneToOne
```

```
    @JoinColumn(name = "pasaporte_id")
```

```
    private Pasaporte pasaporte;
```

```
}
```

@Entity

```
public class Pasaporte {  
    @Id @GeneratedValue  
    private Long id;  
    private String numero;  
}
```

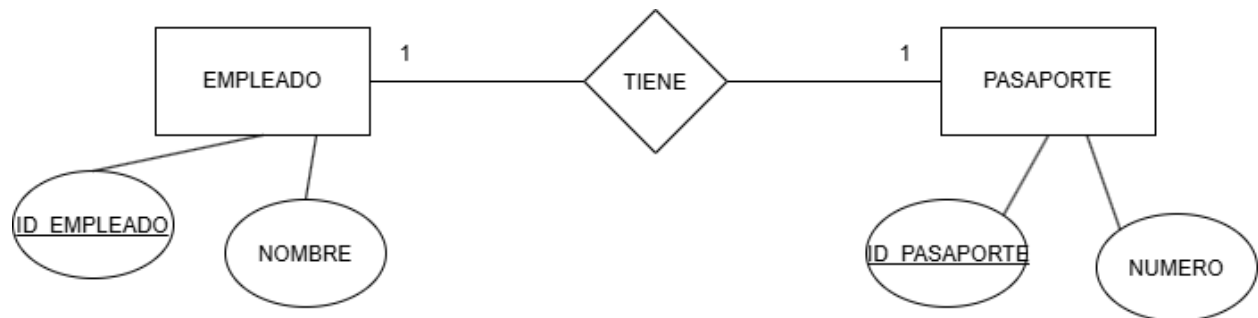


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

- **@OneToMany / @ManyToOne**

(<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#onetomany-annotation> /
<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#manytoone-annotation>)

EJEMPLO:

@Entity

```
public class Departamento {  
    @Id @GeneratedValue  
    private Long id;  
    private String nombre;  
  
    @OneToMany(mappedBy = "departamento")  
    private Collection<Empleado> empleados = new HashSet<>();  
}
```

@Entity

```
public class Empleado {  
    @Id @GeneratedValue  
    private Long id;  
    private String nombre;
```

@ManyToOne

```
    private Departamento departamento;  
}
```

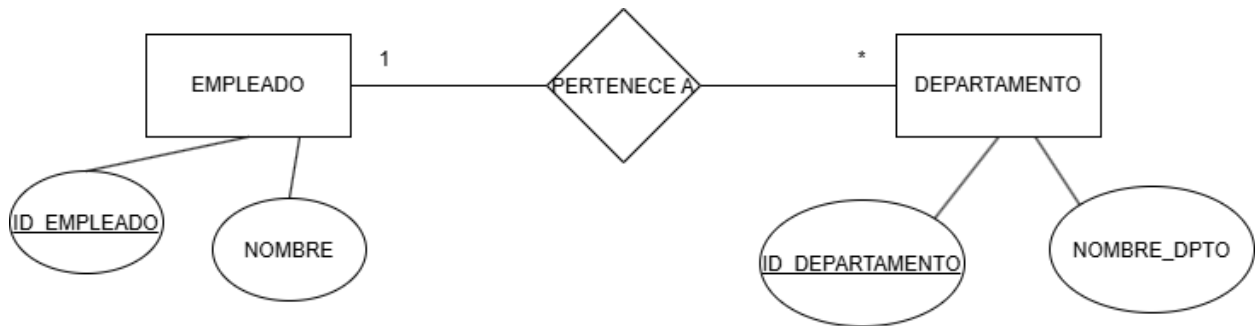


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

- **@ManyToMany** (<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a15100>)

EJEMPLO:

@Entity

```
public class Estudiante {  
    @Id @GeneratedValue  
    private Long legajo;  
    private String nombre;
```

@ManyToMany

```

private Collection<Curso> cursos = new HashSet<>>();
}

```

@Entity

```

public class Curso {

```

```

    @Id @GeneratedValue

```

```

    private Long id;

```

```

    private String nombre;

```

```

    @ManyToMany(mappedBy = "cursos")

```

```

    private Collection<Estudiante> estudiantes = new HashSet<>>();

```

```

}

```

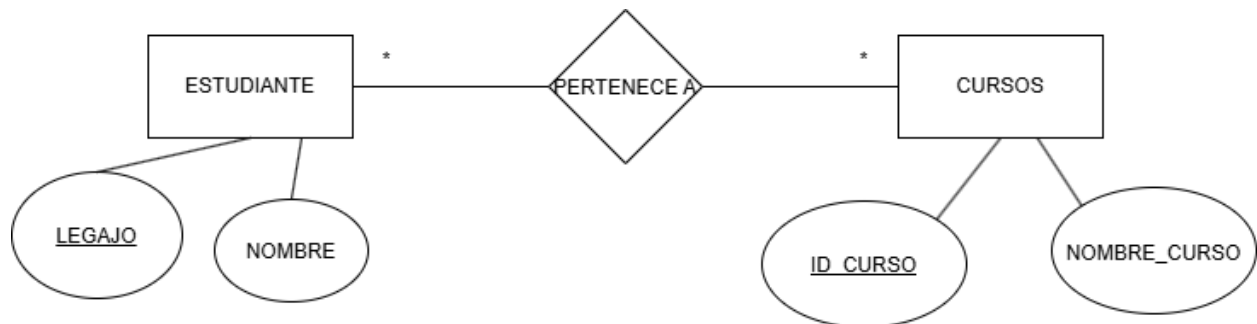


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

¿Graficar paso por paso cómo es el funcionamiento de JPA para el Mapeo de una clase Entidad hasta su conversión en tabla y sus respectivas relaciones?

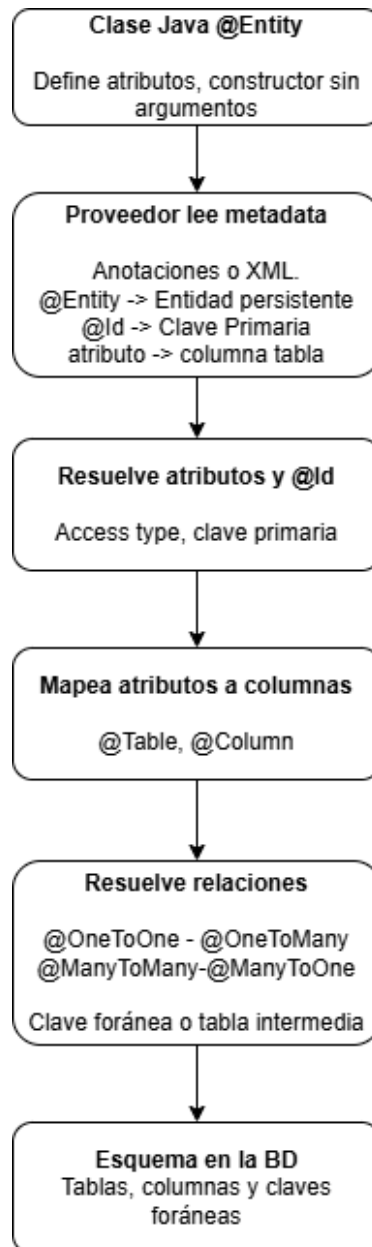


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

¿Qué es un JPAQL? ¿Qué similitudes y que diferencias existen con una sentencia SQL?. Dar 5 ejemplos de JPAQL y su respectiva sentencia SQL.

El lenguaje de consulta JPAQL es un lenguaje de especificación de consultas para consultas dinámicas basadas en cadenas y consultas estáticas expresadas mediante metadatos. Utiliza una sintaxis similar a SQL para seleccionar objetos o valores basándose en tipos de esquemas abstractos y relaciones.

JPAQL se parece mucho a SQL salvo que el primero consulta entidades y atributos Java, mientras que el segundo tablas y columnas; por ende, JPAQL devuelve objetos y SQL registros.

Otra diferencia importante es que JPAQL es independiente de la Base de Datos utilizada, mientras que SQL sí depende del motor que se elija.

EJEMPLOS:

- **OBTENER TODOS LOS CLIENTES**

JPAQL:

```
SELECT c  
FROM Cliente c
```

SQL:

```
SELECT *  
FROM cliente;
```

- **BUSCAR CLIENTES POR NOMBRE**

JPAQL:

```
SELECT c  
FROM Cliente c  
WHERE c.nombre = Jorge
```

SQL:

```
SELECT *  
FROM cliente  
WHERE nombre = 'Jorge';
```


- **BUSCAR PRODUCTOS CUYO PRECIO SEA MAYOR A \$1000**

JPAQL:

```
SELECT p  
FROM Producto p  
WHERE p.precio > 1000
```

SQL:

```
SELECT *  
FROM producto  
WHERE precio > 1000;
```

- **ORDENAR CLIENTES POR APELLIDO**

JPAQL:

```
SELECT c  
FROM Cliente c  
ORDER BY c.apellido ASC
```

SQL:

```
SELECT *  
FROM cliente  
ORDER BY apellido ASC;
```

- **CONTAR LA CANTIDAD DE CLIENTES**

JPAQL:

```
SELECT COUNT(c)  
FROM Cliente c
```

SQL:

```
SELECT COUNT(*)  
FROM cliente;
```

Respuesta encontrada en: <https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a4665> , [Chat.openai.com](https://chat.openai.com) , diapositivas.